ESP8266 实现一键下载固件的深入分析

前言

手头有两块ESP8266的开发板，一块是大家比较熟悉的NodeMCU , 另外一块是espressif 官方的开发板，两块开发板在下载程序的时候，有很大的区别，espressif 官方开发板需要手动以拨动开关的方式去reset 相关引脚，使得esp8266进入boot模式，而nodeMCU 则不需要手动去设置，运行下载脚本时，自动会进入boot模式。为了探究其中的奥秘，笔者查阅了相关资料，并且对其原理进行了剖析，整理了这篇文章，以便给大家在进行ESP8266电路设计时提供一定的参考。

1 ESP8266 BOOT 流程

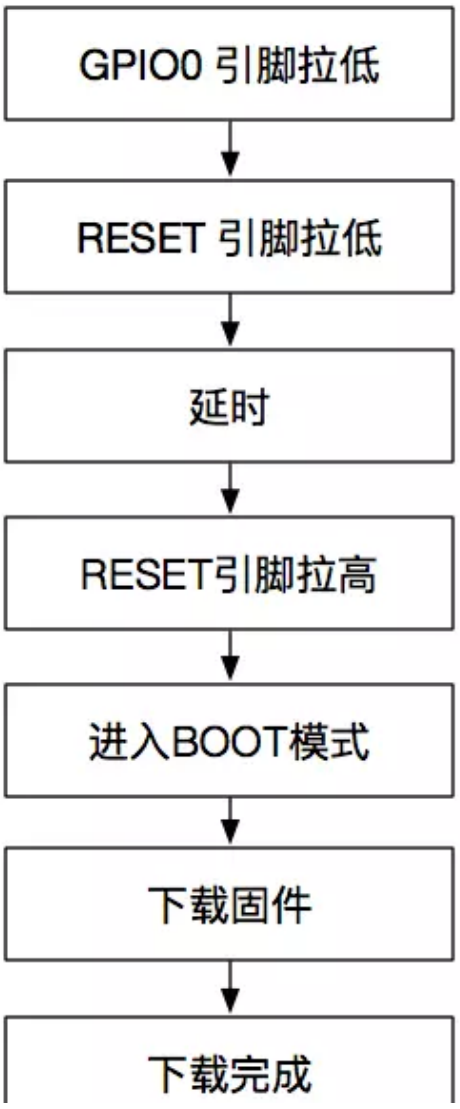
官方资料显示，ESP8266想要进入BOOT模式，必须满足以下条件

1 ） CHIP_EN 引脚 置高

2 ） GPIO0 引脚 置低 （置高则进入从FLASH运行模式）

而在实际设计过程中，CHIP_EN引脚，通常是处于高电平状态.（如果是MCU 控制是否使能WiFi模块， 则建议将CHIP_EN引脚接上拉电阻，使其初始状态为高电平）

而实现一键下载的关键就在于，GPIO0引脚和RESET引脚电平状态，下载流程如下图所示.



ESP8266 下载固件的流程

2 硬件上的改造

下载固件的过程不需要用到串口流控，所以就用不到DTR 和RTS 引脚，而可以通过PC端软件去控制这两个引脚的电平状态，如果利用这两个引脚分别去控制ESP8266的RESET和GPIO0引脚, 在下载过程中按照下载的时序去设置其电平状态，那么就可以实现一键下载。

所以，综上，要从硬件和软件上分别去改造，首先看下硬件上的改造。

上面的电路中，DTR 和RTS 分别来自UART转USB芯片，如CP2102, PL2302， FT232等，

当 DTR为1， RTS为0时， nRST复位引脚拉低，反之，GPIO0 引脚拉低，逻辑关系如下：

AUTO PROGRAM CIRCUIT

DTR	RTS	RST	GPIO0
1	1	1	1
0	0	1	1
1	0	0	1
0	1	1	0

3 软件上的改造

esptools.py是官方提供的一款开源的串口下载工具，我们通过它的源码来分析一键下载的实现。

esptools 中操作串口使用的是pyserial 这个库.

```
135     def connect(self):
136         print('Connecting...')
137
138         for _ in range(4):
139             # issue reset-to-bootloader:
140             # RTS = either CH_PD or nRESET (both active low = chip in res
141             # DTR = GPIO0 (active low = boot to flasher)
142             self._port.setDTR(False)
143             self._port.setRTS(True)
144             time.sleep(0.05)
145             self._port.setDTR(True)
146             self._port.setRTS(False)
147             time.sleep(0.05)
148             self._port.setDTR(False)
```

在connect 方法中, 142行和143行首先将DTR拉低，RTS拉高，结合上面的逻辑图，此时，GPIO0被拉低

延时50ms后，145行和146行，DTR拉高，RTS拉低，结合逻辑图，RESET被拉低，芯片复位，ESP8266进入boot模式。

延时50ms后，148行的作用，RESET和GPIO0恢复到初始状态.

4 结论

综上来看，

首先，实现ESP8266一键下载还是很简单的一个过程，主要是巧妙利用了串口的流控引脚，并且要控制好电平翻转的时机。

其次，这种利用串口流控引脚来实现一键下载的方法同样可以用到其他可以串口BOOT的芯片上，如STM32等。